OpenZeppelin | security

OpenZeppelin SUI Marketplace DApp

April 6, 2026

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Item Listings and Checkout 5

Discount System 5

Oracle Integration and Price Conversion 6

Security Model and Trust Assumptions 6

High Severity 8

H-01 Excessive Price Latency Check Leads to Denial of Service 8

Medium Severity 9

M-01 Ticket Discounts Vulnerable to Sybil Claim Exhaustion 9

Low Severity 10

L-01 Events Lack Contextual Data 10

Notes & Additional Information 11

N-01 Shared Object Transfer Best Practice 11

N-02 Misleading accepted_currency_id in Events 11

N-03 Inconsistent Event Emission Pattern 11

N-04 Incorrect Error for Future-Timestamped Price 12

N-05 Incomplete Documentation 12

N-06 Incorrect Code Comments 13

N-07 Redundant Checks and Storage Operations 13

N-08 Freezing ShopOwnerCap Would Grant Universal Admin Access 14

N-09 Shared Price Feeds Across Accepted Currencies Can Cause Mispricing 15

N-10 Uncapped claims_issued Counter Can Overflow 15

Conclusion 16

Appendix 17

Issue Classification 17

Summary

Type	DeFi
Timeline	From 2026-03-09 To 2026-03-20
Languages	Move

Total Issues	13 (7 resolved, 1 partially resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	1 (1 resolved)
Medium Severity Issues	1 (1 resolved)
Low Severity Issues	1 (0 resolved, 1 partially resolved)
Notes & Additional Information	10 (5 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the OpenZeppelin/openzeppelin-sui-marketplace repository at commit f239ad5.

In scope was the following file:

```
packages
└── dapp
    └── contracts
        └── oracle-market
            └── sources
                └── shop.move
```

Update: *The fixes for the findings highlighted in this report have all been merged at commit f2010a4.*

System Overview

The audited codebase is a marketplace module deployed on the SUI network, implemented as a single Move smart contract. It allows shop owners to list items priced in USD cents, accept payments in multiple cryptocurrencies via real-time Pyth oracle price feeds, and offer configurable discount promotions. The module is built around a shared `Shop` object that stores all listings, accepted currencies, and discount templates in typed dynamic collections (`Table`). It relies on the SUI framework for object ownership, transfers, and coin operations, on Pyth for oracle price data, and on OpenZeppelin's math utilities for fixed-point arithmetic during price conversion.

Item Listings and Checkout

Shop owners register item listings with a base price in USD cents and a finite stock count. Buyers purchase items through one of three checkout paths, all of which perform the same core payment flow:

- Convert the listing's USD price to the buyer's chosen cryptocurrency amount using a Pyth oracle quote, applying configurable guardrails for price staleness and confidence interval.
- Split the payment coin on-chain: the owed amount is sent to the shop owner, and any excess is refunded to a caller-specified address.
- Decrement the listing's stock and mint a phantom-typed `ShopItem<TItem>` receipt as proof of purchase.

The three entry points are `buy_item` for standard purchases, `buy_item_with_discount` for purchases using a previously claimed discount ticket, and `claim_and_buy_item_with_discount` which atomically claims and redeems a discount within a single programmable transaction block.

Discount System

The discount system follows a template-ticket-redemption lifecycle. Shop owners create `DiscountTemplate` entries that define a discount rule (fixed USD cents or percentage in basis points), an activation time window, an optional redemption cap, and an optional listing

scope. Buyers claim a `DiscountTicket` from a template, which is an owned object that can later be burned during checkout to apply the discount. A one-claim-per-address constraint is enforced through a `claims_by_claimer` table, and redemption validates that the ticket's original claimer matches the buyer.

Oracle Integration and Price Conversion

Each accepted currency is bound to a specific Pyth `PriceInfoObject` at registration time, with its feed ID and object ID validated on-chain to prevent feed spoofing. During checkout, the module converts the USD cent price to the target cryptocurrency amount using the oracle's price, exponent, and confidence interval. The conversion uses a conservative price estimate by subtracting the confidence band from the reported price, protecting the seller against price uncertainty. Guardrails for price staleness (default 60 seconds), confidence ratio (default 10%), and attestation lag (default 5 seconds) are set by the shop owner as caps, and buyers may only tighten them further per transaction.

Security Model and Trust Assumptions

The module's security model centers on capability-based authorization via the `ShopOwnerCap` owned object and on the integrity of the Pyth oracle network for price data. Administrative operations are gated by possession of the capability object rather than sender address checks, delegating access control to SUI's object ownership model. The correctness of payment amounts depends entirely on the oracle feed returning accurate and timely price data.

- The holder of the `ShopOwnerCap` is fully trusted to manage the shop. This includes:
- Adding and removing item listings, and updating stock.
- Registering and removing accepted currencies with their oracle guardrail caps.
- Creating, updating, and toggling discount templates, and pruning claim markers after a template is finished.
- Transferring ownership of the payout address and permanently disabling the shop.
- The `ShopOwnerCap` has the `store` ability, making it freely transferable. Loss or theft of the capability object is equivalent to losing control of the shop, as there is no recovery mechanism.

- The Pyth oracle network is trusted to provide accurate price data. The module validates the feed identifier and object identity at currency registration to prevent feed substitution, but cannot independently verify the correctness of the reported prices.
- The shop owner is trusted to associate each accepted currency with a correct Pyth price feed. There is no on-chain validation that a given `feed_id` corresponds to the coin type being registered, so a misconfigured feed would result in incorrect pricing for all purchases settled in that currency.
- A completed purchase does not transfer the actual item to the buyer. Instead, the contract mints a `ShopItem<TItem>` receipt as proof of payment. Fulfillment of the underlying item is outside the scope of this contract.
- Shop disabling is irreversible. Once disabled, all buyer-facing operations are permanently halted while administrative functions remain accessible.
- The oracle guardrail caps set by the shop owner establish a trust boundary: buyers can tighten staleness and confidence parameters per transaction but cannot loosen them beyond the owner-defined ceilings.
- While the system redirects surplus funds to a refund address set by buyers, they are incentivized to provide coin objects with values that closely match off-chain quotes. This is because the module lacks an internal slippage protection mechanism. Therefore, providing specific coin values is the primary method for users to protect themselves against unfavorable price fluctuations during the purchase process.

High Severity

H-01 Excessive Price Latency Check Leads to Denial of Service

One of the checks performed on oracle data enforces a strict five-second maximum difference between the `attestation_time` and the `publish_time` of a Pyth price feed. The `attestation_time` was traditionally set by Wormhole guardians when agreeing on a price, while the `publish_time` reflects when the price was originally computed by the provider. The `assert_price_status_trading!` macro uses this difference to reject feeds that are not actively trading.

However, modern versions of the Pyth accumulator no longer utilize the original attestation timestamp. Instead, the field is populated with the current SUI network timestamp solely for backward compatibility. As a result, regardless of when or by whom the price feed is updated on-chain, the `attestation_time` will always reflect the SUI timestamp of the transaction that last updated the price feed, rather than a guardian attestation. The check then effectively becomes `price_update_timestamp - publish_time <= 5`, comparing the SUI timestamp of the last update transaction against the original off-chain publish time. Given the inherent delivery latency of the Pyth pipeline (Pythnet computation, Hermes relay, and SUI transaction finality), a 5-second window between these two timestamps is expected to be exceeded under normal operating conditions. This creates a denial-of-service condition where purchase attempts are rejected despite using fresh, valid price data.

Consider removing the `assert_price_status_trading!` check entirely, as the `attestation_time` field no longer carries the data it was designed to validate. Price freshness is already enforced by the `max_price_age_secs` guardrail against `publish_time`, which provides the intended protection without relying on a deprecated field.

Update: *Resolved in pull request #101. The OpenZeppelin development team stated:*

> *Addressed on the smart contract side. Additionally field `max_price_status_lag_secs_cap` was removed from `AcceptedCurrency`.*

Medium Severity

M-01 Ticket Discounts Vulnerable to Sybil Claim Exhaustion

The discount system enforces a supply cap through the `max_redemptions` field on `DiscountTemplate` and a one-claim-per-address constraint tracked in `claims_by_claimer`. When a user calls `claim_discount_ticket`, the `assert_template_claimable!` macro verifies that `claims_issued < max_redemptions` and that the sender has not already claimed. Once validated, `record_discount_claim` increments `claims_issued` and marks the address, consuming one of the finite slots.

Because `claim_discount_ticket` is a standalone public function, it allows any address to reserve a discount slot without purchasing an item. The only cost is the transaction fee, which is negligible on SUI. An attacker can exhaust all available slots across many addresses, causing subsequent calls to revert with `ETemplateMaxedOut` and blocking legitimate users from claiming discounts. Although the contract already provides `claim_and_buy_item_with_discount`, an atomic function that claims and purchases in a single call, the existence of the separate claim path makes this protection bypassable.

Consider removing the standalone `claim_discount_ticket` function and routing all discount claims through `claim_and_buy_item_with_discount`. Additionally, `buy_item_with_discount` would need to be removed or refactored so it no longer accepts an external `DiscountTicket`, and the `store` ability on `DiscountTicket` could be dropped since the ticket would no longer need to be transferable. This would ensure that only users willing to pay for an item can consume a discount slot, making the attack economically impractical.

Update: *Resolved in pull request #102. The OpenZeppelin development team stated:*

> *The whole logic associated with `DiscountTickets` and claims was dropped.*

Low Severity

L-01 Events Lack Contextual Data

Most events emitted throughout the contract contain only object identifiers without the associated state changes or values. This limits the usefulness of events for off-chain indexing, analytics, and monitoring. Some examples include:

- `ShopOwnerUpdatedEvent` does not include the old or new owner address.
- `ItemListingStockUpdatedEvent` does not include the previous or updated stock values.
- `DiscountTemplateToggledEvent` does not include the new `active` state. Additionally, `toggle_discount_template` sets the `active` field to the provided value rather than toggling it, meaning the event is emitted even when no state change occurs. This could mislead off-chain components that interpret each event as an actual toggle.
- Discount-related events such as `DiscountClaimedEvent`, `DiscountTemplateCreatedEvent`, and `DiscountTemplateUpdatedEvent` provide no information about the discount rule, amounts, or claimer.

Consider enriching events with the relevant state values (old and new where applicable) so that off-chain consumers can reconstruct state transitions without querying on-chain objects. For `toggle_discount_template`, consider either skipping the event emission when the state does not change, or renaming the function to better reflect its setter behavior.

Update: *Partially Resolved in PR#107.*

Notes & Additional Information

N-01 Shared Object Transfer Best Practice

The `Shop` object is defined with both `key` and `store` abilities. In SUI, the recommended pattern is to use the public version of transfer and share functions for objects that possess the `store` ability.

However, when a new shop is created, it is converted to a shared object using the non-public method.

Consider using `transfer::public_share_object` instead of `transfer::share_object` to align with the recommended patterns for objects that have `store`.

Update: *Resolved in PR#106.*

N-02 Misleading `accepted_currency_id` in Events

The `accepted_currency_id` field in `AcceptedCoinAddedEvent` and `AcceptedCoinRemovedEvent` actually corresponds to the Pyth `PriceInfoObject` ID rather than an identifier for the accepted currency itself. Since these events are the primary interface for off-chain integrators, the naming may cause confusion.

Consider renaming the field to `pyth_price_info_object_id` to clearly convey what the value represents.

Update: *Resolved in PR#105.*

N-03 Inconsistent Event Emission Pattern

Event emissions do not follow a consistent pattern across the contract. For example, `new_discount_template_created_event` is emitted separately in both `add_item_listing_with_discount_template` and `create_discount_template`,

even though both call `create_discount_template_core` in their call path. In contrast, `new_item_listing_added_event` is emitted inside the shared `add_item_listing_core` function rather than in each caller.

Consider centralizing event emissions in shared core functions to reduce duplication and ensure consistent behavior as new entry points are added.

Update: *Resolved in PR#104.*

N-04 Incorrect Error for Future-Timestamped Price

The staleness check in `quote_amount_with_guardrails` reverts with `EPriceTooStale` when `publish_time` is greater than the current clock time. However, this condition indicates a future-timestamped price, not a stale one.

Consider using a distinct error constant to accurately reflect the cause of the revert.

Update: *Resolved in PR#103.*

N-05 Incomplete Documentation

Several functions lack documentation or have undocumented return values:

- The `init` function has no documentation. Its doc comment is placed above the `SHOP` struct instead.
- Return values are not documented for `create_shop`, `add_item_listing_core`, `add_item_listing_with_discount_template`, and `claim_discount_ticket_inline`.
- Several internal functions have no documentation at all, including `link_listing_spotlight_template`, `add_item_listing_with_discount_template_core`, `create_discount_template_core`, `claim_discount_ticket_with_event`, `transfer_shop_owner_cap`, `transfer_discount_ticket_to_recipient`, as well as all data constructors and helper functions.

Consider adding documentation to all functions, including return value descriptions, to improve maintainability and ease onboarding for future contributors.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Documentation will be added after refactoring following audit.*

N-06 Incorrect Code Comments

Several comments in the module header and function documentation contain inaccuracies:

- The events documentation reference points to `docs/08-advanced.md`, which should be `docs/14-advanced.md`.
- The coins documentation references `docs/05-currencies-oracles.md`, which does not exist and duplicates the `docs/09-currencies-oracles.md` reference already present on line 42.
- The `buy_item_with_discount` documentation describes `DiscountTemplate` as a shared object, but it is a table entry stored under the `Shop` object.
- The module header states that some endpoints use `entry` visibility, but no `entry` function exists in the module.
- The fixed-point math comment states discounts are stored in basis points, but discounts can also be stored in cents (fixed amount discounts).

Consider correcting these comments to avoid misleading developers working on the codebase.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Code comments will be updated after refactor following audit.*

N-07 Redundant Checks and Storage Operations

Several checks and storage operations throughout the contract are redundant, increasing gas costs and code complexity without providing additional safety.

Redundant checks:

- `assert_shop_active!` in `claim_and_buy_item_with_discount` is repeated by the subsequently called `buy_item_with_discount`. The same applies to `claim_discount_ticket`, where the inner `claim_discount_ticket_inline` already performs the check.
- The price staleness assertion in `quote_amount_with_guardrails` is superseded by `get_price_no_older_than` on the next line, which performs a strictly tighter check (using `<` rather than `<=`).

- The template-to-shop check in `add_item_listing_core` is performed twice: once inside `assert_listing_inputs` and again in `assert_spotlight_template_matches_listing`. Furthermore, the latter check is unreachable in practice because the listing has not been created yet at that point, so no template can reference it.
- `assert_supported_decimals` in `new_accepted_currency` duplicates the same check already performed by the caller.

Redundant storage:

- The `pyth_object_id` read from the newly created `accepted_currency` object is unnecessary given that `pyth_object_id` is already available as a function parameter.
- The `ID` fields stored on both `ItemListing` and `DiscountTemplate` duplicate the table key used to access them, since callers must already know the key to retrieve these objects.

Consider removing the redundant checks and storage operations to reduce gas overhead and improve code clarity.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Assertion checks will be carefully revised in refactoring following audit.*

N-08 Freezing `ShopOwnerCap` Would Grant Universal Admin Access

The `ShopOwnerCap` struct has both `key` and `store` abilities. While `store` is required for transferability via `transfer::public_transfer`, it also enables `transfer::public_freeze_object`. Freezing the capability would turn it into a globally accessible immutable object. Since all admin functions accept `&ShopOwnerCap` as an immutable reference, a frozen capability would satisfy every authorization check, granting anyone full admin access to the shop. An inexperienced owner might freeze their capability believing it prevents transfers and is therefore safe, without realizing the object becomes publicly readable and usable by all.

Consider documenting this risk for shop owners and integrators.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Documentation will be updated later after audit.*

N-09 Shared Price Feeds Across Accepted Currencies Can Cause Mispricing

Each `Shop` allows registration of multiple accepted currencies for payments, stored in the `accepted_currencies` table and each associated with a `pyth_object_id` used for price discovery. The `add_accepted_currency` function enforces uniqueness only on the coin `TypeName`, not on the `pyth_object_id` or underlying `feed_id`. As a result, multiple distinct coin types with different market values or decimal precisions can be registered under the same price feed.

When two coins share a price feed, the contract treats them as having the same unit value. A buyer would naturally settle with whichever asset is cheaper relative to that feed, causing the shop owner to receive less value than intended. This is a shop owner misconfiguration rather than an external exploit, but the lack of a uniqueness check makes it easy to introduce inadvertently.

Consider enforcing uniqueness of `pyth_object_id` (and optionally `feed_id`) across all entries in the `accepted_currencies` table to prevent accidental mispricing.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Uniqueness check will be added for `pyth_object_id` in the refactoring following the audit.*

N-10 Uncapped `claims_issued` Counter Can Overflow

When a `DiscountTemplate` is created without a `max_redemptions` value, the supply cap check in `assert_template_claimable!` is skipped entirely. Each claim still increments `claims_issued` without any upper bound, which can eventually overflow the `u64` counter. While not practical given the number of unique addresses required, an unbounded counter should not rely on external constraints to prevent overflow.

Consider guarding the increment with an overflow check or enforcing a maximum value for `claims_issued` regardless of whether `max_redemptions` is set.

Update: *Resolved in pull request #102. The OpenZeppelin development team stated:*

> *The `claims_issued` field was removed within M-01 refactor. Now this issue is not applicable.*

Conclusion

The OpenZeppelin SUI Marketplace DApp implements a single-module marketplace on the SUI network, enabling shop owners to list items priced in USD, accept payments in multiple cryptocurrencies through Pyth oracle price feeds, and offer discount promotions via a template-ticket-redemption lifecycle. The module makes effective use of Move's type system and SUI's capability-based authorization model through the `ShopOwnerCap` pattern.

The audit identified one high-severity, one medium-severity, one low-severity, and ten informational findings. The high-severity finding relates to a broken oracle attestation-lag check that renders the checkout flow unusable under normal conditions. The medium-severity finding concerns the ability to exhaust discount ticket supply at negligible cost through the standalone claim path. The low-severity finding covers insufficient event data for off-chain consumers. The informational findings address topics such as the risk of freezing the `ShopOwnerCap` object, potential mispricing from shared price feeds, redundant checks and storage operations, documentation inaccuracies, and naming clarity.

Overall, the codebase was found to be well-structured, with clear separation between administrative and buyer-facing flows and a comprehensive test suite. However, the oracle price conversion logic lacked edge case coverage around boundary conditions, rounding behavior, and realistic price scenarios. Expanding coverage in these areas would increase confidence in the correctness of the pricing arithmetic.

We appreciate the OpenZeppelin development team for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.